



Hochschule RheinMain
Fachbereich Design Informatik Medien
Studiengang Medieninformatik

Abschlussarbeit

zur Erlangung des akademischen Grades

Master of Science

L^AT_EX-Template für schöne Abschlussarbeiten

Vorgelegt von	Robin Kuschnerer
am	21. Juli 2025
Referent	Prof. Dr. Adrian Ulges
Korreferent	Viola Campos

Hinweis: Dieser Text ist wortwörtlich zu übernehmen, mit Ausnahme des grauen Abschnitts. Diesen legen Sie in Absprache mit dem Referent vor Beginn der Bearbeitungszeit fest.

Eigenständigkeitserklärung (gem. ABPO, Ziff. 4.1.5.4 bzw. RPO, §19.6)

Ich bin verantwortlich für die Qualität des Inhalts dieser Arbeit, den ich mit geeigneten wissenschaftlichen Quellen belegt bzw. gestützt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Texte, Gedankengänge, Konzepte, Grafiken, technischen Inhalte und ähnliches in meinen Ausführungen habe ich eindeutig gekennzeichnet und mit vollständigen Verweisen auf die jeweilige Quelle versehen. Alle weiteren Inhalte der Arbeit (Textteile, Abbildungen, Tabellen etc.) ohne entsprechende Verweise stammen im urheberrechtlichen Sinn von mir. Ich versichere außerdem, dass ich die Bachelor-Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Äußern Sie sich hier zur Verwendung generativer KIs. Dieser Abschnitt ist mit dem Referenten abzustimmen. Mögliche Formulierungen sind:

1. Verbot KI-generierter Inhalte: Ich versichere, dass ich die hier vorliegende Arbeit vollständig eigenständig erarbeitet, also keine KI-generierten Inhalte verwendet habe.
2. Erlaubnis KI-generierter Inhalte: Ich versichere, dass ich KI-Tools lediglich als Hilfsmittel verwendet habe und in der vorliegenden Arbeit mein gestalterischer Einfluss überwiegt. Ich verantworte die Übernahme jeglicher von mir verwendeter KI-generierter Inhalte vollumfänglich selbst.
 - (optional, falls Kennzeichnungspflicht) Unter „Übersicht verwendeter Hilfsmittel“ habe ich im Anhang der Arbeit sämtliche KI-generierten Inhalte einzeln aufgeführt und die jeweils verwendete KI-Anwendung benannt.
 - (optional, falls Prompts abzugeben) Die jeweiligen Benutzungseingaben (z.B. Prompts) sind der Arbeit beigelegt.

Auch hiervon abweichende Regelungen sind in Absprache mit dem Referenten möglich.

Die vorliegende Arbeit wurde bisher weder im In- noch im Ausland in gleicher oder ähnlicher Form einer anderen Prüfungsbehörde vorgelegt. Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte als Täuschungsversuch gelten und prüfungsrechtliche Konsequenzen haben kann. Insbesondere kann es dazu führen, dass die Leistung nicht bestanden ist und dass bei mehrfachem oder schwerwiegendem Täuschungsversuch eine Exmatrikulation droht.

Wiesbaden, 21. Juli 2025

Robin Kuschnereit

Erklärung zur Verwendung der Masterthesis

Hiermit erkläre ich mein Einverständnis mit den im folgenden aufgeführten Verbreitungsformen dieser Abschlussarbeit:

Verbreitungsform	Ja	Nein
Veröffentlichung des Titels der Arbeit im Internet	×	
Veröffentlichung der Arbeit im Internet		×

Wiesbaden, 21. Juli 2025

Robin Kuschnereit

Abstract

Contents

1	Introduction	4
2	Foundations	6
2.1	Large Language Models	6
2.2	Calibration	6
2.3	Metrics	6
2.3.1	ECE	6
2.3.2	Brier score	6
2.3.3	ASCE	6
2.3.4	GASCE	6
2.3.5	Skill score	7
2.4	Methods	7
2.5	Related Work	7
3	Concept	8
3.1	Data generation	9
3.2	Evaluation through a LLM	12
3.3	Calibration foundations	13
3.3.1	Discretizing scores	14
3.3.2	Visualizing calibration result	14
3.4	Calibration	15
3.4.1	Histogram binning	15
3.4.2	Linear regression	16
3.4.3	Iterative group histogram binning	16
3.4.4	Iterative group linear binning	17
3.4.5	Regressor	18
4	Evaluation	20
4.1	Data	20
4.2	Models	20
4.3	Results	21

4.4 Experiments	21
5 Discussion	22
Sources	23
A Prompts	25
A.1 Evaluator LLM	25

Todo list

■ Einleitung: Jeweils Statistiken herausuchen und Belege	4
■ Einleitung: Relevanz großer Sprachmodelle	4
■ nutzung ki in firmen	4
■ Einleitung: Themengebiet Codegenerierung	4
■ Einleitung: Kalibrierung	4
■ Einleitung: Beitrag der Arbeit	4
■ Grundlagen: Survey, bzw Related Work	6
■ Formeln quellen (Primärquelle suchen?)	6
■ Code: Median prompt in code einbauen	12
■ Code: grouping style zwei nur komplexität nutzen in evaluation	12
■ in Bullet points verwandeln	13
■ Pseudo code	17
■ Hauptteil: Formeln für SVR	19
■ Hauptteil: XGBoost SVR besser erklären	19
■ Datensätze beschreiben (1)	20
■ Hier vllm beschreiben und welche Modelle konkret	20

Chapter 1

Introduction

Large language models or short LLM like Chat-GPT are currently a big topic. This models are also used by software developers [9]. The developer can use the LLM to generate code through interaction. Furthermore the LLMs can also try to fix faulty code or complete existing code. The focus of this work lies on this two subject areas. Code generation and program repair.

That generated code is used is not a rarity anymore. A study finds that approximately 31% of postdocs use AI-assistants like Chat-GPT [4].

Furthermore the work from Vaithilingam et al. [9] shows that generated code might not be perfect but can be a good starting point for further development. The AI-assistant builds the rough framework and the developer refines the generated code.

After the generation of code the model can give you a confidence as indicator. But this confidence is not a good indicator to find out if the generate code is functioning or if the model might made some mistakes [6]. Such mistakes could be unreachable code, logical errors or security issues.

For evaluation of the question if generated code can be trusted is reliable metric useful. This metric should display how confident the model is in the generated code. It is assumed that the confidence score or the token likelihoods are not enough, because the models tend to overestimate or underestimate their results. This fact should be improved by using calibration. To achieve this we use different calibration methods. This methods were already use by Detommaso et al. [2], but in their case the used these methods for hallucination detection on question-answer datasets.

Goal of this work is to develop a model that can calibrate the confidence values of a LLM. This confidence should represent the expected probability of the correctness after calibration. This means that if a sample has a confidence of 50% it has a probability of 50% to be correct. This allows the user to identify the trustworthiness of the generated code.

For such a model exist different approaches. This approaches are shown and discussed in this work. Our own contribution here is the realization of the calibration and multicalibration approaches onto the subject areas code generation and program repair. Furthermore we look into the subject of creating groups. These groups are used in the multicalibration approaches. Here we look into which groups are useful to better the calibration on code generations. To create this groups we use metadata of the generated code. Another difference is the fact that we look into the topic of correctness prediction instead of hallucination. That is what Detommaso et al. [2] did in their work.

This work is structured as follows. In the first chapter 2 the foundations for our work are explained. This includes the basics of LLM, calibration, important metrics and the calibration methods we use. Furthermore we display works that are related to our topic. The next chapter 3

Einleitung: Jeweils Statistiken
heraussuchen und Belege

Einleitung: Relevanz großer
Sprachmodelle

nutzung ki in firmen

Einleitung: Themengebiet Code-
generierung

Einleitung: Kalibrierung

Einleitung: Beitrag der Arbeit

covers an overview over our used components. These components include the data generation, evaluation of code with a LLM and the calibration approaches. Then the evaluation chapter 4 displays the results of our work. Therefore we show the used datasets, benchmarks and models. Furthermore we conduct experiments with the different approaches. This experiments will all answer a question. In the discussion chapter 5 we set our results into context and will display potential weaknesses. Besides that we give some options for further research.

Chapter 2

Foundations

Grundlagen: Survey, bzw Related Work

2.1 Large Language Models

2.2 Calibration

2.3 Metrics

Formeln quellen (Primärquelle suchen?)

2.3.1 ECE

$$ECE = \sum_{k=1}^n P_k * |correctness_k - confidence_k| \quad (2.1)$$

2.3.2 Brier score

The brier score or also called mean squared error (MSE) is defined as follows:

$$MSE = \frac{1}{n} \sum_{k=1}^n (label - confidence)^2 \quad (2.2)$$

2.3.3 ASCE

$$ASCE = \sum_{k=1}^n P_k * (correctness_k - confidence_k)^2 \quad (2.3)$$

2.3.4 GASCE

$$GASCE = \sum_{k=1}^n P_{k,g} * (correctness_{k,g} - confidence_{k,g})^2, \text{ where } g \in G \text{ and } n \text{ is the bin count} \quad (2.4)$$

2.3.5 Skill score

To specify the skill score we first need to introduce the brier reference score which is defined by formula 2.5. To calculate the brier reference score we need the probability that a sample is correct $P_{correct}$ over the whole dataset

$$\text{brier ref} = P_{correct} * (1 - P_{correct}) \quad (2.5)$$

$$\text{skill score} = \frac{\text{brier ref} - MSE}{\text{brier ref}} \quad (2.6)$$

2.4 Methods

2.5 Related Work

This section present different works that were made in the last years which ave the subject of evaluating LLM generated code.

The work from Spiess et al. [7] deals with the calibration and correctness of code LLM. They argue that these LLM can oft be helpful, but often generate faulty code. They have found out that the from them tested LLM (CodeGen2, Codex, GPT-3.5) are not well calibrated out of the box. This means that the confidence of the LLM does not correlate well with the probability of correctness of the generated results. Furthermore they show how to improve the calibration with platt scaling, which helps to better classify the results.

The work from Ni et al. [6] deals with the ability of LLM to convert language to code. The found that the models which perform best are not the ones that are best calibrated. Better model are according to Ni et al. [6] more reliable, because the calibrated confidence can be used as a good indicator to evaluate the generated code.

Vaithilingam et al. [9] conducted a user study whit software developers. These developers where equipped with the GitHub Copilot assisted to solve programming tasks. The found out that the GitHub Copilot generated correct code in most cases. But the could not discover a significant time saving. They attribute this results to the fact that the developers still need to check the code for correctness and if they find bus they need to first fully understand it to fix the code. Nevertheless the generated code can be helpful to serve as a starting point.

Virk et al. [10] investigate the calibration of confidence score for natural language code summaries. They create a confidence score which mirrors the similarity of the a code summary from a human. To calibrate their results the used platt scaling. They found that there results provide reliable results for the predictions.

The work from Zhong and Wang [11] suggest a new kind of dataset to evaluate LLM generated code. This dataset enables the evaluation of the reliability and robustness of LLM generated code which interacts with APIs. They argument that previous dataset and benchmarks are restricted to small programming tasks, which does not reflect the everyday programming tasks. Even GPT-4 achieves a misuse of 62% of API interactions. This can lead to huge damage in real world applications.

Chapter 3

Concept

x = sample
 X = all samples
 y = label of correctness $\{0, 1\}$
 Y = all labels
 $g(x)$ = group function $\{0, 1\}$
 G = all groups
 $P(x)$ = raw uncalibrated probability (confidence)
 $P'(x)$ = discretized probability
 $\hat{P}(x) = P_u(x)$ = updated probability
 w = weight for group in linear regression
 m = number of bins
 $grid = [\frac{1}{m}] = \{0, \frac{1}{m}, \frac{2}{m}, \dots, \frac{m-1}{m}, 1\}$
 $d(x)$ = function to assign bin to sample
 $S_b = \{x | d(x) \in grid\}$
 $S_{b,g} = \{x | d(x) \in grid \wedge g(x) \in G\}$
 $\tau = \tau$ = IGLB CDF function, $0 \rightarrow \leq, 1 \rightarrow \geq$
 $S_{\tau,b,g} = \{x | d(x) \in grid \wedge g(x) \in G \wedge \tau \in \{0, 1\}\}$
 $\delta_b = \delta_b$ = delta to update bin
 $\delta_{b,g} = \delta_{b,g}$ = delta to update bin-group combination
 α = parameter in IGHB approach
correctness per bin??
 $P(y = 1 | S_b, b \in grid)$
confidence ??
 $P(S_b)$

Our goal is to create a model that gets a raw uncalibrated probability and input features for a code generation and outputs a well calibrated probability. Such a well calibrated probability helps the software developer to better interpret the code generations and serves as a indicator for possible bugs. In Diagram 3.1 all main components that are used in our pipeline are shown. The left side shows the code and data generation and the right side displays the calibration part. The process starts with the code generation prompt. This prompt is put into an LLM to generate a program. Besides the generated code we also fetch metadata like the token scores. We process this data to get a raw uncalibrated probability for each generated sample. This serves as a baseline to compare our calibrated probabilities against. After the generation, we evaluate the samples for correctness. This is done by using test cases. After the evaluation, we form groups. This groups are used to summarize samples which have the same properties. For example one group could contain each sample that is written in the programming language python. The concept of groups is used in some of our calibration approaches. Their purpose is to only correct samples that are in a certain group. Therefore we achieve better calibration in

this groups.

Furthermore, we will explore an optional process which involves another LLM. This LLM is used to evaluate the generated code. The LLM evaluation output can either be treated directly as calibrated scores or be used for further calibration. After these step we use the outcoming data for calibration.

After the preparation phase we come to the main part of our work where we can choose between several calibration methods. These methods are histogram binning (HB), linear regression (LR), iterative group histogram binning (IGHB) or iterative group linear binning (IGLB). We measure how well the calibrated scores align with the true probability of a code sample to be correct before each method is started and after the method was used. Besides using every method on their own its also possible to use every method at once. This is used to compare the outputs of each method.

The methods histogram binning, linear regression, iterative group histogram binning and iterative group linear binning are used from the paper multicalibration for confidence scoring in LLMs from Detommaso et al. [2]

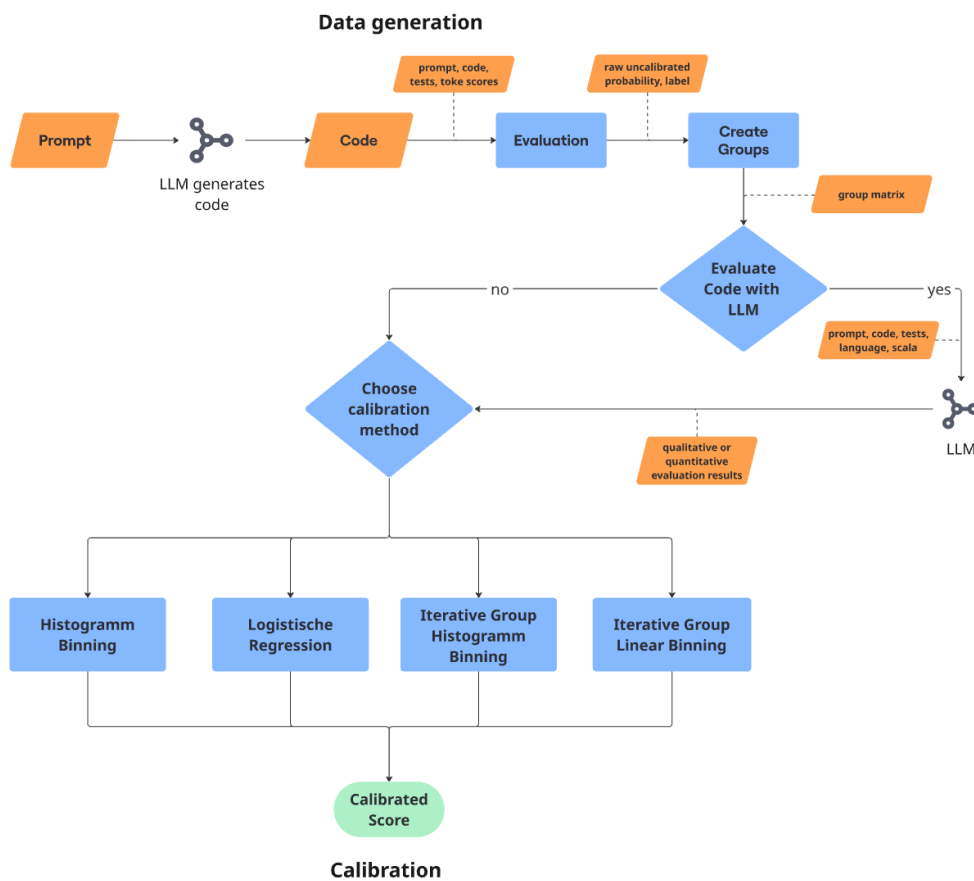


Figure 3.1: Calibration process flowchart

3.1 Data generation

At the data generation beginning we got a prompt. This prompt specifies the criteria for the code to be generated. Depending on the dataset, this prompt consists of imports, a method hull and a doc string. In the method hull we find the function name, input parameters with their data type and the expected output data type. The doc string contains a description that

specifies the functionality which is expected of the code. In most cases the prompt also contains examples. The examples specify cases for which inputs and outputs are expected. In Listing 3.1 we can see an example prompt from the humaneval dataset. This prompt is written in python.

Listing 3.1: Humaneval python task 19 example prompt [1]

```

1 from typing import List
2
3 def sort_numbers(numbers: str) -> str:
4     """ Input is a space-delimited string of numerals from 'zero' to
5     'nine'.
6     Valid choices are 'zero', 'one', 'two', 'three', 'four', 'five',
7     'six', 'seven', 'eight' and 'nine'.
8     Return the string with numbers sorted from smallest to largest
9     >>> sort_numbers('three one five')
10    'one three five'
11    """

```

The prompt is forwarded to a LLM. The LLM then tokenizes, embeds the prompt and predicts the method body. This is done with the selected decoding strategy.

For the generation we use vllm [3]. Vllm is a fast and easy to use framework to conduct inferences with LLMs. Specific LLMs can be selected through vllm's so called model. This can be any model from huggingface hub or a downloaded model. For the generation we pass the prompts into vllm's generate() method. This method also takes sampling parameters. These specify parameters like the temperature, number of generations, stop words, max tokens and logprobs. Which parameters we used during the experiment will be specified in the evaluation Chapter 4.

As response from the LLM, we get the generated token sequences. These sequences contain the completed code pieces. Besides these sequence we additionally fetch details about each token. This data contains an entry for each generated token. This entry contains the token id, the actual log probability, the rank and the actual decoded token. Furthermore, we fetch the cumulated token probability which sums up all token probabilities. Besides the information about the output tokens we also gather information about the input tokens. These data has the same form like the output data. Through that we can regenerate the probabilities for a given code generation.

In this work we used open source models. These model allow easy access to token probabilities and we can control what the influencing variables are. These token probabilities indicate how confident the model is in the selection of the token. Note that the returned probabilities are log probabilities and need to be converted into normal probabilities to allow an intuitive assessment. In Figure 3.2 we can see a correct generation for the prompt from Listing 3.1. We can see that the generation was cut off at the first not intended comment. The colors indicate the confidence of the model in the token: a high confidence is colored dark green and a low confidence is colored in red.

See highlighted code

Generated code:

```

# A dictionary to map numerals to their corresponding digits
num_map = {
    'zero': '0', 'one': '1', 'two': '2', 'three': '3', 'four': '4',
    'five': '5', 'six': '6', 'seven': '7', 'eight': '8', 'nine': '9'
}

# Split the input string into words and map each word to its corresponding digit
digits = [num_map[num] for num in numbers.split()]

# Sort the list of digits and map them back to numerals
sorted_digits = sorted(digits)
sorted_num_map = {num_map[num]: num for num in num_map}
sorted_numbers = ' '.join(sorted_num_map[digit] for digit in sorted_digits)

return sorted_numbers
#

```

Figure 3.2: Highlighted tokens with color that indicates confidence

The above generation process is applied to every sample in the given dataset, obtaining one generation for each prompt. To summarize, each sample has a prompt, the generated completed code, and details about the token sequence most importantly the individual token scores.

After the generation process we check the correctness of the generated code completions. This is done by applying test cases that are provided with each sample by the respective research dataset. The test cases check if the generated code produces the correct output for a given set of input values. To do so we concatenate the prompt with the generated code completion and append the test cases. Resulting in a program that we can execute. Depending on the result of the execution it is specified if the sample is correct or incorrect: if its executable and passes all test cases the sample gets the label correct. This label is defined as y . Fails the sample test cases or is not even executable it is labeled as incorrect. We mention at this point that test cases can have blind spots and may not cover every possible input-output combination.

To summarize, we got at this point for every sample the program that consists of the prompt, code completion and the tests, the raw uncalibrated probability and the label which indicates if the sample is correct.

For some calibration methods we discuss later we need features that describe the sample. We call these features groups. A group is a subset of the dataset. These groups can also be intersecting. For example a piece of generated code is written in the language python and has examples in the initial prompt. We could assume that LLMs have a worse structural correctness and therefore calibration with different kind of programs. Maybe they have better calibration on samples which contain examples and a worse calibration on sample with long input prompts. That would be intuitively the case for humans.

The groups are associated with the samples through a matrix. Let N be the number of samples and M be the number of groups. Then the matrix is $N \times M$. This matrix contains zeros and ones which display the affiliation of the sample and the groups. Each row is a sample and each column represents a group. A zero means that the sample does not belong to this group and a one means it does. This matrix is built during the data loading process and will be used in different calibration approaches in the next sections. A sample matrix is displayed in Figure 3.3. The columns G_n stand for the groups and the row S_n stand for the samples. The set G contains

all groups.

$$\begin{matrix} & G_1 & G_2 & \dots & G_m \\ \begin{matrix} S_1 \\ S_2 \\ \\ S_n \end{matrix} & \begin{bmatrix} 0 & 1 & \dots & 1 \\ 1 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \dots & 0 \end{bmatrix} \end{matrix}$$

Figure 3.3: Group matrix

In this work we use three different approaches to create groups. The first grouping style we introduce is named „simple“. This grouping style is our the entry point into the group topic. We use simple characteristics to build the groups matrix. The criteria we check for are: the prompt contains the word "example", is the prompt longer than the median prompt, and is the program code longer than the median program code. The idea behind the example group is that it should intuitively be easier to create a program if you have examples that show how the code works. The same goes for the group that checks the prompt length. Here we we assume that the problem becomes more complex the longer the prompt is. For the last group, which checks the length of the generated program, we assume that longer generated code is more likely to be incorrect than shorter code.

Our second grouping style approach is called „complexity“. For this we analyze the generated programs. This is done with command line tool called „scc“ which was created by Ben Boyter [12]. This tool outputs the lines of code, lines of comments and an approximation of cyclomatic complexity per program. The cyclomatic complexity is a metric that measures the independent paths through a program and was introduced by Thomas McCabe [5]. It should be noted that the complexity score we use is only an approximation because its difficult to automate the calculation over multiple different program languages because of their differences in programming paradigms [12]. We choose to create a group for different complexity levels as defined by Thomas McCabe [13]. The first group contains all samples that have a cyclomatic complexity of 1-10. These samples are considered to have a small risk. The second group contains all samples that have a complexity of 11-20 and have moderate risk. The third group contains all samples with a complexity of 21-50 and have a high risk. The last group contains all samples that have >50 complexity and have a very high risk. These four groups contain all samples and are therefore self-contained.

The third grouping style places the samples into groups which represent semantic categories of programming problems. These categories are extracted from the task definition that is contained in the prompt. To achieve this, we check the prompt for certain keywords and add them to the group. The first group is build to represent tasks that use mathematical operations and numbers. This assignment takes place through the check for keywords like 'integer', 'float' and a combination of 'number' and 'calculate'. The second group represents all samples that operate with strings or chars. Therefore we search for keywords like 'string', 'char' or 'word'. The last group contains all samples that use higher level data objects like lists. Here we check for keywords like 'list', 'array' or 'tree'.

Code: Median prompt in code einbauen

Code: grouping style zwei nur
komplexität nutzen in evalua-
tion

3.2 Evaluation through a LLM

One straightforward approach comes to mind to estimate our desired probability

$P(\text{correct}|\text{generatedcode})$: It would be to simply ask another LLM if the generated code is correct. Such auto-evaluation is quite common in question, answering, for example <https://autoevaluator.langchain.com/>.

Description	Propability
Very low	0
Low	0.3
Somewhat low	0.45
Medium	0.5
Somewhat high	0.65
High	0.7
Very high	1

Table 3.1: Qualitative scale mapping

We also created a method to evaluate the generated code with the help of a LLM. The LLM can either be the same LLM or we can choose another one. This is helpful if want to evaluate the generated code with a more powerful LLM. To get an evaluation of the generated programs, we build a prompt which contains the programming language, the program, which contains prompt, generated code and test cases, and an instruction to evaluate the correctness of the code. The actual prompt templates can be found in the appendix A.1. The prompts and approaches are inspired by the work of Ulmer et al. [8]. Note that the evaluator LLM does not need to be a open source model and it is also not necessary to retrieve the log probabilities. This is possible because we directly use the generated token sequences from the evaluator LLM and do not use the log probabilities.

For the evaluation of the generated code we differentiate between a quantitative and a qualitative approach.

in Bullet points verwandeln

The quantitative approach tries to directly get a percentage of correctness from the model. Therefore the model is asked to return a percentage of correctness from the range of 0 to 100. This percentage of correctness is then treated as the confidence in the correctness of the sample. We assume that it is hard for a model to rate the program by using a continuous number range.

The qualitative approach, on the other hand, gives the model the option to choose from a preset of qualitative descriptions. For example „High“ or „Very low“. These qualitative scale can be found in Table 3.1. Based on the selection the model makes the sample gets assigned a specific percentage. These values are defined independently and can also be adjusted after the model made its prediction.

For both methods we use template matching to extract the values from the answer token sequences. For the qualitative approach we search for a number with a percentage symbol and for the qualitative approach we search for the predefined qualitative descriptions. Therefore we use the first occurrence.

The probability outputs of these method can than be used for further calibration methods in the process. It is also possible to not use any calibration method and analyze the pure outputs from the evaluator LLM.

3.3 Calibration foundations

For our calibration approaches we need a probability which reflects the confidence in the generated code. We obtain a first baseline estimate of this confidence from the individual token probabilities. We assume that if the model is „insecure“ about its generation these toke score will be low / have a low entropy. With them we calculate the average token log probability per sample. This is done my accumulation of the token log probabilities. This number is then divided by the number of generated tokens. The result is then transformed into actual probabilities by using the exponential function. The formula is displayed in 3.1. We assume

that these average probabilities are the confidence of the model in the generated code for further experiments. Lower average token probabilities imply an uncertainty in the model output. Higher confidence scores imply that the model thinks that its output is more reliable. As a results of this process step, we get an probability for each sample that represents the confidence.

$$P(x) = \exp\left(\frac{\sum_{k=1}^n \log prob_k}{n}\right) \quad (3.1)$$

These uncalibrated probabilities of our code samples are the baseline for our further work. We will use them to verify if we can achieve a better calibration with the calibration methods we display in the following sections.

3.3.1 Discretizing scores

To review the calibration on our generated data, we need to split the probability range into bins. This is done by splitting the range into equally large bins. They are defined by $[\frac{1}{m}] = \{0, \frac{1}{m}, \frac{2}{m}, \dots, \frac{m-1}{m}, 1\}$. We control the number of bins with the parameter m . To assign this probabilities to a bin we need to know which probability comes into which bin. To achieve this we calculate the distance to each bin and choose the bin which is closest to the probability. The formula to calculate said assigned bin can be seen in 3.2.

$$P'(x) = \operatorname{argmin}(|probability - bin|), \text{ where } bin \in [\frac{1}{m}] \quad (3.2)$$

This mapping is then used to calculate the correctness in each bin. This is done before and after the calibration method was applied. Through that we can evaluate the calibration on the uncalibrated data, the calibration on the calibrated data and the increase or decrease of metrics that were showed in Chapter 2.

3.3.2 Visualizing calibration result

To compare the approaches we calculate metrics on the uncalibrated data and then after each method was used. Through this we get an idea of how well each method performed under the same circumstances. We can now use we use reliability charts and histograms to plot the changes that each calibration method applies. The reliability charts show each bin and the correctness of the samples in said bin. The histograms show the distribution of the samples in the grid and can therefore track where the distribution mass moved. In Figure 3.4 we can see an example for a reliability chart. On the x-Axis we got the confidence values that we received from the LLM. On the y-Axis we got the probability that a sample is correct for each bin. The blue diagonal line symbolizes a perfect calibration. The bar colors of the reliability diagram display the sample distribution. Darker blue shades mean more samples are in this bin and light blue or white displays less samples. Furthermore the numbers about the bars show the actual count of samples in this bin.

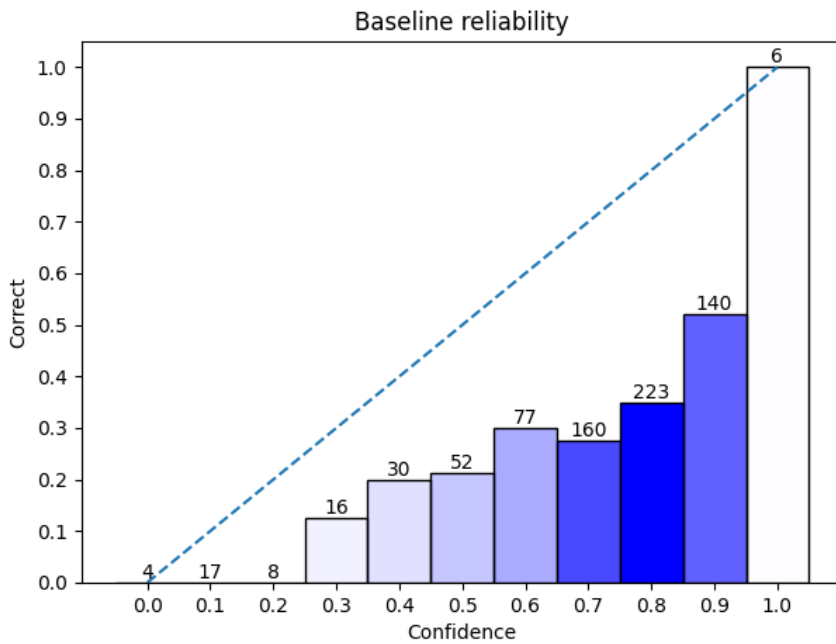


Figure 3.4: Reliability chart to display calibration

In Figure 3.4 we can see the uncalibrated data distribution of a dataset. As we can see the probabilities are unevenly distributed and are not close to the diagonal line which would be the perfect calibration of the underlying data. Perfectly calibrated data would mean that in bin 0.5 are always 50% correct and 50% incorrect samples. The bin with a confidence 0.8 contains 223 samples. It is the bin with the most samples and there for has the darkest shade of blue. The raw uncalibrated probabilities express a high confidence (80%). But we can see that samples in this bin are have a correctness of around 35%. This is an example of poor calibration. For a good calibration the bar should be at 80%.

3.4 Calibration

A calibration method is something that gets the raw uncalibrated probabilities and turns them into better calibrated probabilities. We hope that these better calibrated probabilities provide a better estimation of the correctness of a generated code sample. Some of these methods do not only take the raw uncalibrated probabilities, but also use groups. These groups are used to obtain a better calibration on the subset of samples that are contained in this groups. We hope to achieve better calibration by choosing a good combination of groups.

3.4.1 Histogram binning

The first calibration approach is called histogram binning (HB). This approach is the simplest of those studied in this thesis and tries to adjust all probabilities in a bin at once. To achieve the calibration improvement, we calculate the difference between the correctness and the average confidence in the bin for all defined bins. These so called deltas in formula 3.3 are learned under supervision on a trainings set. Because we discretized the values in each bin the average confidence is always the value of the bin. For example in the bin 0.5 the average confidence is 0.5. The correctness is calculated by using the label of the samples. With the these two values we calculate the difference (delta) between optimal calibration and actual calibration. This is

done for every bin.

$$\text{delta}_b = \text{correctness}_b - \text{confidence}_b, \text{ where } b \in \left[\frac{1}{m}\right] \quad (3.3)$$

The calculated deltas are then used to update all discretized probabilities in each bin. These updates are made to every defined bin. This is done by adding the corresponding delta with the discretized probabilities in the selected bin. The result of this are new adjusted probabilities for every sample. This process shifts the probabilities to a better calibrated version. But it neglect the fact that different bins and the contained samples are not always disjoint and do maybe depend on each other [2].

3.4.2 Linear regression

The second method we use to calibrate our raw uncalibrated probabilities is linear regression (LR). In this approach the groups appear. The groups are used as input features for the linear regression. In formula 3.4 these groups are displayed as x_i and can take the value one or zero. As output we want the difference between the label and the actual probability which is displayed as y . The label defines if a sample is correct or not and therefore can take the values zero or one. Which means incorrect sample will be shifted into the direction of zero because they become a negative sign and the correct sample will be shifted more to the direction of one because of the positive sign.

$$y = b + w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n \quad (3.4)$$

To realize this the linear regression learns the weights w_n for each group and the bias b . In this approach we use the actual probabilities and not the discretized values. Further more the bias b of the linear regression is used to adjust samples that have no groups associated with. These weights and the bias are learned on a training set.

With the learned weights $w_1, \dots, w_n$ and bias b we predict the adjustment for a given group feature input $x_1, \dots, x_n$. The predicted y value is then added to the raw uncalibrated probability of the sample. $\text{new probability} = y + \text{probability}$. Through that we get a better calibrated probability for the sample.

We also conducted a control experiment in which the actual labels are learned. The results will be shown in Chapter 4.

3.4.3 Iterative group histogram binning

Iterative group histogram binning (IGHB) is a approach which combines the idea of groups with the concept of bins. Instead of calculating deltas for each bin, like we did it in the histogram binning 3.4.1 approach, we calculate these deltas for every bin-group combination. That is done by calculating the difference between correctness and the average confidence for every bin and the groups in the given bin as we see in Formula 3.5. Through this we create a matrix of delta values. Where M is the number of bins and G is the number of groups. Then we get matrix with the shape of $M \times G$.

$$\text{delta}_{b,g} = \text{correctness}_{b,g} - \text{confidence}_{b,g}, \text{ where } b \in \left[\frac{1}{m}\right] \text{ and } g \in G \quad (3.5)$$

The problem that arises is that we now have multiple delta values for a sample. That is because a sample can be in multiple groups, but only can be in one bin. So instead of applying the deltas to every sample in our dataset, like we did with histogram binning, we select the combination of bin and group which has the most impact on the calibration. To calculate this we square our

deltas 3.5 and multiply them with the probability that a sample is in the bin-group combination. For the formula see 3.6 By doing this we get the bin and group which has the highest impact on the calibration of the data.

$$(b, g) = \operatorname{argmax}(\operatorname{delta}_{b,g}^2 * P(b \cap g)), \text{ where } b \in [\frac{1}{m}] \text{ and } g \in G \quad (3.6)$$

The adjustments of the sample probabilities are made in a loop. This loop stops if the maximal error in the data is smaller then a predefined value α . This α value also defines the grid size in this approach. The α value of 0.1 results in a grid of 10 bins. So the grid is defined by calculating $\frac{1}{\alpha}$ and using this value as the bin width. The max error on the other hand is recalculated on each iteration of the loop. It is defined as the result of multiplying the GASCE with the probability of which a sample is in a given group as we can see in formula 3.7.

$$\operatorname{error} = \operatorname{argmax}(\operatorname{GASCE} * P(g)), \text{ where } g \in G \quad (3.7)$$

In each iteration we adjust the probabilities on all subsets of our data. For evaluation purposes we track the changes which each iteration made. After each iteration we fit the calibrator with the adjusted probabilities. That means we calculate each deltas again and check if the error is still greater than the α value. Is that not the case we stop the adjustment and the calibration process is finished.

Through the specific adjustment we make for the bin-group combinations there is a high risk of overfitting on the data. That is caused by the small amount of data that is adjusted in each iteration.

To apply the learned changes onto an new sample we need to keep track of every adjustment that is made per iteration. Therefore we need to save the bin-group combination that was adjusted and the delta which was applied. It is important to mention that these changes need to be applied in the exact same order as they where saved. That is because through the changes of time samples are moved into other bins and could be affect by the next change in another way.

Pseudo code

3.4.4 Iterative group linear binning

Iterative group linear binning (IGLB) improves the previous IGHB approach. Therefore three adjustments are made. First we not only apply the corrections for exactly one bin-group combination but to a cumulative bins instead of individual ones. This set is defined by the parameter τ . A τ of zero means we adjust all values that are smaller than or equal the selected bin. Analog a τ of on means we adjust all values that are greater than or equal the selected bin. The parameter τ specifies this cumulative distribution function. The update of a larger number of samples is assumed to help to limit the risk of overfitting.

Secondly we apply so called linear scaling within each bin. This scaling is defined by the formula 3.8:

$$P'(x) = \operatorname{expit}(\alpha + \beta * \operatorname{logit}(P(x))) \quad (3.8)$$

X are the probabilities of the samples and X' are the new adjusted probabilities. To get an optimal set of alpha and beta values we minimize the MSE on Formula 3.8. This is done by using the Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm. As initial start parameter we use 0 for alpha and 1 for beta. By applying the formula we calculate the alpha and beta values for all tau-bin-group combinations. In the prediction phase we select the set which is needed for the adjustment.

We choose the subset we correct by calculating the probability that a sample is in a given subset with parameter tau, group and bin. We also do this analog with the delta values. This

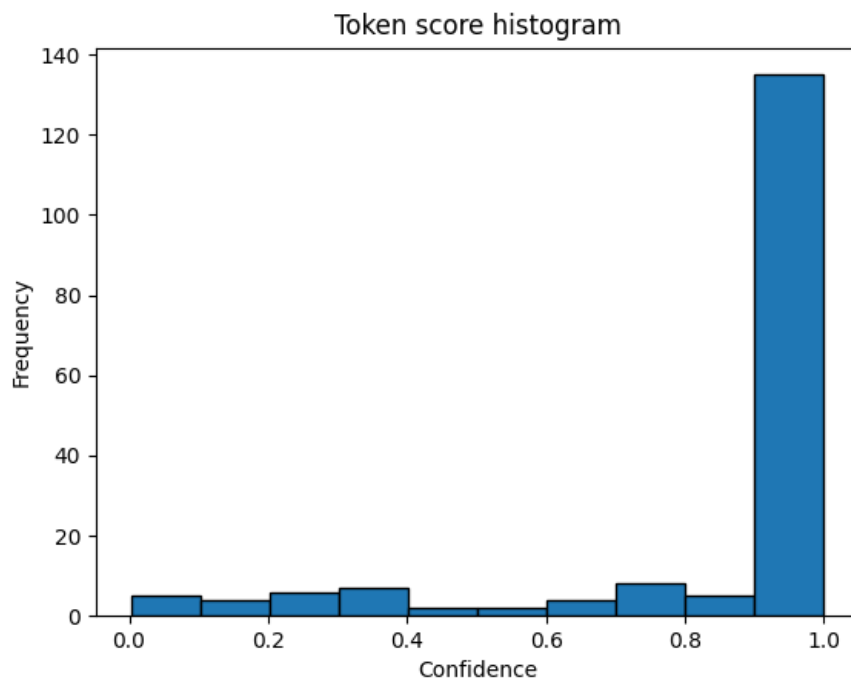


Figure 3.5: Histogram that displays the distribution of the token confidence in the humaneval task 19

This input features were used to train the regressor. As target we predict the deltas to adjust the probabilities for better calibration. As regressor we use linear regression, support vector regression (SVR) and XGBoost.

The linear regression is used like described in Section 3.4.2, but with the modified input features.

For the SVR we use the radial basis function and for XGBoost we used the default settings to fit our data. All three regressors get the same input features and same target value. We input our features and learn the deltas for calibration. For the prediction part we apply the outputs of the regressor to the sample probability. We assume that the regressors learn deltas that improve the calibration of our data. At this point no bin assignments are made for the calibration process.

Hauptteil: Formeln für SVR

Hauptteil: XGBoost SVR besser erklären

Chapter 4

Evaluation

4.1 Data

Datensätze beschreiben (1)

As data for our experiments we mainly use the humaneval dataset [1]. This data consists of 164 programming tasks.

For this work we also investigate the generalization abilities of the different calibration approaches. Therefore we train the approaches on 60% of the underlying data. The other 40% are used for validation (20%) and testing (20%) of the learned parameters. We decided to use cross validation only in certain cases because the data is sufficiently large enough.

The output of each method is stored in a folder structure. This folder structure is created while loading the for the calibration approach. It changes dynamically when the approach is called with another parameter set over the command line. The following is a example how the structure looks: „runs/humaneval-all-keep-Qwen2.5_Coder_7B-Instruct-1.0-comp-1/lr/linear/avg_logprob/categories/split“. The runs folder contains all results that are generated. The second folder is the run which describes the dataset and for code generation also the model with which the samples where generated. After this the calibration approach is specified in this case its „lr“ and under this folder we find the different binning types. For this work we only considered linear binning so its the only possible option. The next option specifies which method is used to generate the probability which is associated with the sample. The options are the average token probability (avg_logprob), the quantitative or the qualitative probabilities which where generated by the evaluation LLM. After this we got folder which specify the grouping style which were explained in the section 3.1. The last folder contains either the results for the partitioned data or for the run with all data and no split into train, test and validation.

In the deepest folder we find charts which display the calibration before and after the method was used and a table which contains the calculated scores. Furthermore there is a file which shows the date at which time the data in this folder was generated.

4.2 Models

Hier vllm beschreiben und welche Modelle konkret

4.3 Results

4.4 Experiments

Chapter 5

Discussion

Sources

Literaturverzeichnis

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.
- [2] Gianluca Detommaso, Martin Bertran, Riccardo Fogliato, and Aaron Roth. Multicalibration for confidence scoring in llms, 2024.
- [3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the 29th Symposium on Operating Systems Principles, pages 611–626, 2023.
- [4] Fabrizio Le. How chatgpt is transforming the postdoc experience. Nature, 622:655, 2023.
- [5] T.J. McCabe. A complexity measure. IEEE Transactions on Software Engineering, SE-2(4):308–320, 1976.
- [6] Ansong Ni, Pengcheng Yin, Yilun Zhao, Martin Riddell, Troy Feng, Rui Shen, Stephen Yin, Ye Liu, Semih Yavuz, Caiming Xiong, Shafiq Joty, Yingbo Zhou, Dragomir Radev, Arman Cohan, and Arman Cohan. L2ceval: Evaluating language-to-code generation capabilities of large language models. Transactions of the Association for Computational Linguistics, 12:1311–1329, 10 2024.
- [7] Claudio Spiess, David Gros, Kunal Suresh Pai, Michael Pradel, Md Rafiqul Islam Rabin, Amin Alipour, Susmit Jha, Prem Devanbu, and Toufique Ahmed. Calibration and correctness of language models for code. arXiv preprint arXiv:2402.02047, 2024.
- [8] Dennis Ulmer, Martin Gubri, Hwaran Lee, Sangdoo Yun, and Seong Oh. Calibrating large language models using their generations only. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 15440–15459, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [9] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In Extended Abstracts of the 2022 CHI Conference on Human Factors in

Computing Systems, CHI EA '22, New York, NY, USA, 2022. Association for Computing Machinery.

- [10] Yuvraj Virk, Premkumar Devanbu, and Toufique Ahmed. Enhancing trust in llm-generated code summaries with calibrated confidence scores. arXiv preprint arXiv:2404.19318, 2024.
- [11] Li Zhong and Zilong Wang. Can llm replace stack overflow? a study on robustness and reliability of large language model code generation. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 38, pages 21841–21849, 2024.

Online-Quellen

- [12] Ben Boyter. Sloc cloc and code (scc). [letzter Zugriff: 6. Jun. 2025].
- [13] Thomas McCabe Jr. Software quality metrics to identify risk. [letzter Zugriff: 6. Jun. 2025].

Appendix A

Prompts

A.1 Evaluator LLM

"Answer as short as you can. Here is a snippet of generated code consisting of the prompt, generated code and test cases: {code}"

Verbalized quantitative prompt: "Please provide your confidence in the correctness of the provided code only in percent (0-100 %): "

Verbalized qualitative prompt: "Please provide your confidence in the correctness of the provided code only as one of {Qualitative scale:}: "

Qualitative scale: "
{
"Very low": 0,
"Low": 0.3,
"Somewhat low": 0.45,
"Medium": 0.5,
"Somewhat high": 0.65,
"High": 0.7,
"Very high": 1,
}"